Guía Gamificada deU1T2 25 Minutos

Modelado con PlantUML: 2 diagramas de casos de uso nivel 0 + 1 diagrama de clases de análisis

Caso: aplicativo móvil para seguimiento a egresados ESPE – IASA-I

Dirigido a	Estudiantes de 6to nivel de la carrera de Software con base previa en Ingeniería de Requisitos.
Modalidad	Trabajo colaborativo en equipos de 3 a 4 integrantes.
Producto final	Dos diagramas de casos de uso nivel 0 y un diagrama de clases de análisis construidos con sintaxis PlantUML.

Idea central del reto: traducir requisitos funcionales del proyecto a modelos UML claros, trazables y coherentes, usando PlantUML como herramienta de representación textual.

1. Propósito del taller

Adaptar la guía gamificada al modelado de artefactos de análisis para el proyecto de seguimiento a egresados. El equipo trabajará como si fuera una célula de análisis responsable de transformar necesidades del caso en modelos UML iniciales listos para discusión técnica.

2. Base conceptual del reto

El caso del proyecto plantea una necesidad institucional: mantener contacto continuo con egresados, actualizar información personal, difundir oportunidades, apoyar capacitación y recopilar retroalimentación para la mejora académica.

- Desde Ingeniería de Requisitos, el taller se enfoca en tres preguntas clave:
  - ¿Quién interactúa con el sistema? → actores.
  - ¿Qué objetivos persigue cada actor? → casos de uso.
  - ¿Qué conceptos del dominio participan y cómo se relacionan? → clases de análisis.

3. Objetivos de aprendizaje

Lo que ya conocen	Lo que practicarán hoy	Evidencia esperada
Actores y necesidades	Identificación de actores primarios y secundarios	Listado depurado de actores
Requisitos funcionales	Conversión de RF a casos de uso nivel 0	2 diagramas de CU coherentes
Trazabilidad	Relación entre objetivo del negocio y funcionalidad	Justificación breve por equipo
Análisis de dominio	Extracción de entidades, atributos y relaciones	1 diagrama de clases de análisis

4. Modelos que debe construir cada equipo

- Diagrama 1: Casos de uso nivel 0 – Actor Socio (Egresado).
- Diagrama 2: Casos de uso nivel 0 – Actor Unidad de Seguimiento / Administrador (Secretario, Presidente, Tesorero).
- Diagrama 3: Diagrama de clases de análisis del dominio.

Sugerencia didáctica: cada diagrama debe responder a una parte clara del problema. No se trata de dibujar todo, sino de modelar lo esencial de la primera versión del sistema.

5. Mecánica gamificada

Insignia	Cómo se obtiene	Puntos
Radar de Actores	Identifica actores correctos y evita redundancias.	3
Arquitecto de Casos de Uso	Construye 2 diagramas nivel 0 con nombres claros y relaciones válidas.	5
Modelador del Dominio	Propone clases de análisis pertinentes con relaciones coherentes.	5
Sintaxis PlantUML Segura	Escribe código PlantUML legible y consistente.	3

6. Secuencia del taller (25 minutos)

Tiempo	Fase	Actividad del equipo	Resultado esperado
0-3 min	Activación	Asignar roles: coordinador, analista de requisitos, modelador PlantUML y vocero. Leer el caso base.	Equipo organizado y comprensión inicial del sistema.
3-8 min	Ronda 1	Identificar actores del sistema (Socio, Secretario, Presidente, Tesorero) y agrupar funcionalidades según sus objetivos.	Lista clara de actores y objetivos asociados.
8-14 min	Ronda 2	Construir el diagrama de casos de uso nivel 0 orientado al Socio, identificando relaciones <<include>> y <<extend>>.	Diagrama CU nivel 0 – Actor Socio estructurado correctamente.
14-19 min	Ronda 3	Construir el diagrama de casos de uso nivel 0 orientado a los actores administrativos (Secretario, Presidente, Tesorero).	Diagrama CU nivel 0 – Actores administrativos completo y coherente.

19-23 min	Ronda 4	Analizar los casos de uso y extraer clases del dominio, atributos y relaciones principales.	Diagrama de clases de análisis consistente con los CU.

Tiempo	Fase	Actividad del equipo	Resultado esperado
23-25 min	Cierre	Validar coherencia entre requisitos, actores y clases. Exposición breve.	Síntesis y retroalimentación.

7. Desarrollo del reto por rondas

Ronda 1. Radar de actores y objetivos

Preguntas guía aplicadas al caso:

- ¿Quién usa el sistema? → Socio
- ¿Quién administra o supervisa? → Secretario, Presidente, Tesorero
- ¿Qué acciones realiza cada uno? → Consultar eventos, proponer eventos, validar propuestas, gestionar información

Resultado del análisis:

- Actor principal: Socio
- Actores secundarios: Secretario, Presidente, Tesorero

Objetivos identificados:

- Socio: consultar eventos, recibir información, proponer eventos
- Administradores: validar, supervisar y gestionar eventos

Errores evitados:

- No se confundieron actores con funcionalidades
- No se duplicaron actores con distintos nombres
- No se modelaron pantallas como casos de uso

Ronda 2. Casos de uso nivel 0 – Actor Socio

Objetivos identificados:

- Socio: consultar eventos, recibir información, proponer eventos
- Administradores: validar, supervisar y gestionar eventos

Errores evitados:

- No se confundieron actores con funcionalidades
- No se duplicaron actores con distintos nombres
- No se modelaron pantallas como casos de uso

Ronda 3. Casos de uso nivel 0 – Actores administrativos

Casos de uso identificados:

- Consultar calendario de eventos
- Recibir notificaciones de propuestas
- Validar requisitos mínimos (<<include>>)
- Gestionar propuestas de eventos
- Revisar detalles de eventos

Ronda 4. Diagrama de clases de análisis

Clases extraídas del dominio:

- Usuario
- Evento
- PropuestaEvento
- Notificacion
- Recordatorio
- CalendarioController

Enumeraciones:

- TipoEvento
- EstadoEvento

Relaciones identificadas:

- Usuario crea PropuestaEvento
- Usuario consulta Evento
- PropuestaEvento genera Notificacion
- Evento tiene Recordatorio
- Evento pertenece a TipoEvento
- CalendarioController gestiona Evento

8. Caso base del taller

Caso: El sistema de gestión de eventos está diseñado para permitir a los socios consultar un calendario de actividades, visualizar detalles, filtrar eventos por tipo y configurar recordatorios, además de proponer nuevos eventos adjuntando información relevante. Paralelamente, los actores administrativos como el secretario, presidente y tesorero supervisan el proceso mediante la recepción de notificaciones, validación de requisitos y gestión de propuestas, asegurando que los eventos cumplan con las condiciones necesarias antes de su ejecución. Este sistema facilita la organización, comunicación y control eficiente de eventos dentro de la institución.

9. Entregable del equipo

- Lista breve de actores identificados (Socio, Secretario, Presidente, Tesorero).
- Código PlantUML del diagrama de casos de uso nivel 0 – Actor Socio.
- Código PlantUML del diagrama de casos de uso nivel 0 – Actores administrativos (Secretario, Presidente, Tesorero).
- Justificación oral de 30 a 40 segundos sobre la trazabilidad entre requisitos y modelos.

10. Plantilla rápida para completar en clase

Actores principales	
Casos de uso del Socio	
Casos de uso de la Unidad / Administrador	
Clases candidatas del dominio	
Relaciones principales entre clases	
Observaciones de trazabilidad	

11. Mini sintaxis PlantUML

Caso de uso nivel 0 – Actor Socio

```
@startuml
left to right direction

actor "Socio" as socio
actor "Secretario" as secretario
actor "Presidente" as presidente
actor "Tesorero" as tesorero

rectangle "Sistema de Gestión de Eventos" {
    usecase "Consultar calendario de eventos"
as UC1
    usecase "Filtrar por tipo
(social/deportivo)" as UC2
    usecase "Configurar recordatorio" as UC3
    usecase "Ver detalle del evento" as UC4

    ' El filtro es opcional, por eso es extend
    UC2 ..> UC1 : <<extend>>

    ' El recordatorio es opcional, por eso es
extend
    UC3 ..> UC1 : <<extend>>

    ' Ver el detalle es parte del flujo de
consulta
    UC1 ..> UC4 : <<include>>
}

socio --> UC1
secretario --> UC1
presidente --> UC1
tesorero --> UC1

@enduml
```

Diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0

class CalendarioController {
    + visualizarMensual(mes: int, anio: int):
List<Evento>
    + aplicarFiltro(tipo: TipoEvento):
List<Evento>
}

class Evento {
    - idEvento: int
    - nombre: String
    - descripcion: String
    - fecha: Date
    - hora: Time
    - ubicacion: String
    - tipo: TipoEvento
    + obtenerDetalles(): String
}

class Recordatorio {
    - idRecordatorio: int
    - anticipacion: int
    - medio: String
    + programar(idUsuario: int): void
    + cancelar(): boolean
}

enum TipoEvento {
    SOCIAL
    DEPORTIVO
}

' Relaciones
CalendarioController ..> Evento : gestiona >
Evento "1" -- "0..*" Recordatorio : puede tener >
Evento "*" -- "1" TipoEvento : categorizado por >
@enduml
```

Caso de uso nivel 0 – Actores Administrativos

```
@startuml
left to right direction

actor "Socio" as socio
actor "Secretario" as secretario

rectangle "Sistema de Gestión de Eventos" {
    usecase "Proponer nuevo evento" as UC_PROP
    usecase "Adjuntar documentos/propuesta" as
UC_ADJ
    usecase "Notificar a directiva" as UC_NOTIF
    usecase "Validar requisitos mínimos" as
UC_VAL

    ' El socio inicia la propuesta
    socio --> UC_PROP

    ' Al proponer, el sistema SIEMPRE valida y
notifica
    UC_PROP ..> UC_VAL : <<include>>
    UC_PROP ..> UC_NOTIF : <<include>>

    ' Adjuntar archivos es opcional
    UC_ADJ ..> UC_PROP : <<extend>>

    ' El secretario recibe la notificación (actor
secundario)
    UC_NOTIF -- secretario
}
@enduml
```

Diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0


class Usuario {
    - id: Long
    - nombre: String
    - correo: String
    - rol: Rol
    + iniciarSesion(): void
}

class Evento {
    - id: Long
    - nombre: String
    - descripcion: String
    - fecha: Date
    - lugar: String
    - tipo: TipoEvento
    - estado: EstadoEvento
    + mostrarDetalle(): String
}

class PropuestaEvento {
    - id: Long
    - motivo: String
    - justificacion: String
    - estado: String
    - adjuntoPath: String
    + enviarParaRevision(): boolean
    + validarRequisitos(): boolean
}

class Notificacion {
    - id: Long
    - mensaje: String
    - fechaEnvio: DateTime
    + enviarNotificacion(): void
}

enum TipoEvento {
    SOCIAL
    DEPORTIVO
}

enum EstadoEvento {
    PROGRAMADO
    EN_CURSO
    FINALIZADO
}

' Relaciones
Usuario "1" -- "*" Evento : consulta >
Usuario "1" -- "*" PropuestaEvento : crea >
PropuestaEvento "1" -- "1" Notificacion :
genera >
Evento "*" -- "1" TipoEvento : tiene >

@enduml
```

12. Criterios de evaluación rápida

Criterio	Qué se observa	Puntaje
Comprensión del caso	Los actores y objetivos corresponden al problema del proyecto.	5
Calidad de casos de uso	Los 2 diagramas nivel 0 son claros, consistentes y sin redundancias.	5
Calidad del diagrama de clases	Las clases de análisis representan el dominio y sus relaciones esenciales.	5
Uso de PlantUML y comunicación	La sintaxis es legible y la explicación del equipo es técnica y breve.	5

13. Cierre sugerido del docente

Mensaje de cierre: modelar bien no es dibujar mucho, sino representar con precisión lo que el sistema debe hacer y los conceptos clave que lo sostienen. Si los requisitos están claros, PlantUML permite convertirlos en artefactos trazables, discutibles y técnicamente útiles.

Documento adaptado para el caso de seguimiento a egresados ESPE – IASA-I y orientado al uso de PlantUML en Análisis y Diseño de Software.